

Projet Ymmo – Plateforme de Gestion immobilière

Rapport de la partie Développement (DEV)

Bachelor 2 Ynov Informatique – UF INFRA & DEV

Introduction

Ymmo est un groupe immobilier implanté en France, avec un siège social basé à Aix-en-Provence et un réseau d'agences réparties sur le territoire national. L'entreprise est spécialisée dans la vente et l'achat de biens immobiliers résidentiels et professionnels.

Dans ce contexte, Ymmo souhaite disposer de sa propre plateforme web centralisée pour :

- Gérer les biens (vente et location),
- Suivre les clients et les transactions,
- Exploiter les données du portefeuille pour guider les décisions d'achat et de vente.

Ce rapport présente la **partie Développement (DEV)** du projet réalisé dans le cadre de l'UF INFRA & DEV du Bachelor 2 Ynov Informatique. Il décrit les besoins fonctionnels, l'architecture technique, la modélisation des données, le fonctionnement du backend et du frontend, ainsi que le script d'analyse de données mis en place.

Cahier des charges fonctionnel (DEV)

1. Gestion des biens immobiliers

La plateforme doit permettre à Ymmo de gérer un portefeuille de biens immobiliers. Un bien est décrit par :

- un titre (ex : "Appartement haussmannien – 6e, vue Tour Eiffel"),

- une ville,
- un prix,
- un statut (à vendre, vendu, à louer, loué),
- une image (URL),
- une adresse complète,
- une surface en m²,
- un nombre de chambres,
- un nombre de salles d'eau,
- un type (appartement, maison, villa, loft, studio, etc.),
- une description textuelle,
- éventuellement des coordonnées GPS (latitude, longitude).

Fonctionnellement, l'application doit permettre :

- de créer un nouveau bien,
- de lister tous les biens,
- de consulter le détail d'un bien,
- de mettre à jour un bien,
- de supprimer un bien.

2. Gestion des clients

La plateforme doit aussi gérer des **clients** de type acheteur ou vendeur. Pour chaque client, on enregistre :

- le prénom,
- le nom,
- l'email,
- le téléphone,
- le type (acheteur ou vendeur).

Fonctionnalités attendues :

- créer un client,
- lister les clients,
- récupérer le détail d'un client,

- supprimer un client.

3. Gestion des transactions

Une transaction représente une **opération réelle** entre un client et un bien. Pour chaque transaction, on stocke :

- le bien concerné (property_id),
- le client concerné (client_id),
- le prix de la transaction,
- la date,
- le type d'opération (vente ou location),
- un statut (par exemple terminée).

Fonctionnalités :

- créer une transaction,
- lister les transactions,
- consulter le détail d'une transaction,
- supprimer une transaction.

4. Logique métier sur les statuts des biens

Un point clé du cahier des charges est de relier les opérations métiers aux statuts du portefeuille. Lorsqu'une transaction est créée :

- si operation_type = "vente", alors le bien doit passer automatiquement au statut vendu,
- si operation_type = "location", alors le bien doit passer automatiquement au statut loué.

Ainsi, l'état du portefeuille (biens disponibles, vendus, loués) reste cohérent avec les opérations réellement effectuées.

2.5. Interface web pour les agents

La solution doit offrir une **interface web** destinée aux agents Ymmo, avec :

- une visualisation du portefeuille de biens sous forme de cartes,
- une carte géographique (OpenStreetMap via Leaflet) affichant les biens par ville ou coordonnées GPS,
- un formulaire pour ajouter rapidement un nouveau bien,
- une fiche détaillée par bien, accessible au clic.

2.6. Analyse et exploitation des données

Enfin, le sujet demande une partie “traitement et analyse de données”.

L’objectif est d’être capable de :

- calculer des **prix moyens par ville**,
- identifier les villes les plus chères,
- identifier les biens les plus grands,
- suivre le **chiffre d’affaires** généré par les transactions (notamment les ventes).

Un script Python d’analyse est donc prévu pour exploiter la base ymmo.db.

Architecture technique

1. Technologies utilisées

Pour répondre au cahier des charges, les choix techniques suivants ont été faits :

- Backend :
 - Framework FastAPI (Python) pour exposer une API REST moderne, typée, avec documentation automatique via Swagger.
 - Utilisation des modèles Pydantic pour la validation des données.
- Base de données :
 - SGBD SQLite, adapté pour un projet pédagogique et une application en local.
 - ORM SQLAlchemy pour gérer les modèles et les requêtes SQL de manière objet.
- Frontend :
 - HTML5, CSS3, JavaScript vanilla (sans framework lourd).
 - Carte interactive basée sur Leaflet et OpenStreetMap pour l’affichage géographique des biens.
- Analyse de données :
 - Script Python autonome analysis.py utilisant le module standard sqlite3 pour se connecter à la base ymmo.db et calculer différents indicateurs statistiques.

2. Organisation du code

L’architecture logique du projet côté DEV est la suivante :

- app/main.py :
 - crée l’application FastAPI,

- configure la connexion à la base,
- monte les différents routers (properties, clients, transactions, stats).
- app/models.py :
 - contient les modèles SQLAlchemy :
 - Property (table properties),
 - Client (table clients),
 - Transaction (table transactions).
- app/schemas.py :
 - définit les schémas Pydantic pour les entrées / sorties :
 - PropertyCreate, PropertyRead,
 - ClientCreate, ClientRead,
 - TransactionCreate, TransactionRead.
- app/routers/ :
 - properties.py : endpoints /properties/,
 - clients.py : endpoints /clients/,
 - transactions.py : endpoints /transactions/ + logique métier sur les statuts,
 - stats.py : endpoint /stats/ pour quelques statistiques simples.
- app/database.py :
 - gestion de la session SQLAlchemy et de la création de la base SQLite.
- frontend/index.html :
 - interface web principale (formulaire d'ajout, affichage des biens, carte Leaflet, fiche détaillée).
- analysis.py :
 - script d'analyse avancée (statistiques sur les biens, clients et transactions).

Modélisation des données

1. Entité Property (bien)

L'entité Property représente un bien immobilier.

Principaux champs :

- id : identifiant unique,

- title : titre du bien,
- city : ville,
- price : prix demandé ou loyer,
- status : statut métier (a_vendre, vendu, a_louer, loue),
- image_url : URL de l'image principale,
- address : adresse complète,
- surface : surface en m²,
- bedrooms : nombre de chambres,
- bathrooms : nombre de salles d'eau,
- type : type de bien (appartement, maison, villa, loft, studio, etc.),
- description : description textuelle libre,
- lat / lng : latitude et longitude pour la carte.

Un bien peut être lié à **plusieurs transactions** au cours de sa vie (par exemple plusieurs locations successives).

2. Entité Client

L'entité Client représente un client Ymmo.

Champs principaux :

- id,
- first_name, last_name,
- email,
- phone,
- type : acheteur ou vendeur.

Un client peut être **acheteur** sur une transaction et **vendeur** sur une autre, selon les cas.

3. Entité Transaction

L'entité Transaction représente une opération effectuée entre un client et un bien.

Champs :

- id,
- property_id : référence vers un Property,
- client_id : référence vers un Client,
- price : prix de la transaction,
- date : date au format chaîne (YYYY-MM-DD),

- operation_type : "vente" ou "location",
- status : par exemple terminee.

La combinaison de operation_type et price permet ensuite de calculer le **chiffre d'affaires** et d'analyser la répartition ventes / locations.

4. Relations

Les relations sont les suivantes :

- Un **bien** (Property) possède **0..N transactions** (Transaction).
- Un **client** (Client) possède **0..N transactions** (Transaction).
- Chaque **transaction** est liée à exactement **1 bien** et **1 client**.

Fonctionnement du backend (API FastAPI)

1. Endpoints principaux

Les principaux endpoints exposés par l'API sont :

- /properties/ :
 - GET /properties/ : retourne la liste de tous les biens,
 - POST /properties/ : crée un bien,
 - GET /properties/{id} : retourne le détail d'un bien,
 - PUT /properties/{id} : met à jour un bien,
 - DELETE /properties/{id} : supprime un bien.
- /clients/ :
 - GET /clients/ : liste des clients,
 - POST /clients/ : création d'un client,
 - GET /clients/{id} : détail,
 - DELETE /clients/{id} : suppression.
- /transactions/ :
 - GET /transactions/ : liste de toutes les transactions,
 - POST /transactions/ : création d'une transaction,
 - GET /transactions/{id} : détail,
 - DELETE /transactions/{id} : suppression.
- /stats/ :

- endpoint dédié à des indicateurs globaux (nombre de biens, etc., selon implémentation).

Grâce à FastAPI, tous ces endpoints sont automatiquement documentés et testables via l'interface Swagger disponible sur /docs

2. Logique métier dans POST /transactions/

La route POST /transactions/ est au cœur de la logique métier :

1. Validation des données d'entrée via le schéma Pydantic TransactionCreate.
2. Vérification de l'existence du **bien** (property_id) dans la base.
3. Vérification de l'existence du **client** (client_id) dans la base.
4. Vérification métier : price doit être strictement positif.
5. Vérification de operation_type : doit être "vente" ou "location".
6. Création de la transaction en base.
7. Mise à jour du statut du bien associé :
 - si operation_type == "vente" ⇒ property.status = "vendu",
 - si operation_type == "location" ⇒ property.status = "loué".
8. Sauvegarde en base et renvoi de la transaction créée au client (front ou API).

Cette logique permet de **synchroniser automatiquement l'état du portefeuille** avec les opérations réalisées. Lorsqu'un bien est vendu ou loué, son statut change dans l'API, et ce changement est ensuite visible sur le site (badges, affichage, etc.).

Interface web (frontend)

1. Structure générale de la page

La page principale index.html est organisée comme un **dashboard d'agent immobilier** :

- Un header avec le logo Ymmo et une courte présentation.
- Un “hero” mettant en avant l'objectif de la plateforme (portefeuille de biens, expérience client).
- Deux colonnes principales :
 - **Colonne gauche** : carte Leaflet,
 - **Colonne droite** : formulaire d'ajout + portefeuille de biens.

2. Formulaire d'ajout de bien

Le formulaire permet d'ajouter un bien avec tous les champs importants :

- Titre, ville, prix, adresse,
- Surface, chambres, salles d'eau,
- Type de bien,
- Statut initial (à vendre / à louer),
- URL de l'image,
- Latitude et longitude (optionnelles),
- Description.

Lors de la soumission :

- les données sont envoyées en POST à l'API /properties/ via JavaScript (fetch),
- en cas de succès, la liste des biens est rechargée et le nouveau bien apparaît dans le portefeuille et sur la carte.

3. Affichage du portefeuille de biens

La colonne de droite affiche une **grille de cartes** représentant chaque bien :

- image du bien,
- titre et adresse,
- ville + type de bien,
- surface, chambres, salles d'eau,
- prix affiché de manière lisible,
- badge indiquant le **statut** (À vendre, Vendu, À louer, Loué) avec un style différent pour les biens non disponibles (gris, badges "vendu" / "loué").

Ces cartes sont construites dynamiquement à partir de la réponse de GET /properties/.

4. Fiche détaillée et carte Leaflet

Au clic sur une carte, une **fiche détaillée** s'ouvre sous forme de modal :

- grande image en haut,
- titre, adresse et prix,
- description complète (ou message "Aucune description..." si le champ est vide),
- bloc "Caractéristiques" (type, surface, chambres, salles d'eau),
- bloc "Disponibilité" :
 - indique si le bien est disponible à la vente/location ou déjà vendu/loué,
 - se base directement sur le status renvoyé par l'API.

La carte Leaflet :

- affiche un marker pour chaque bien,
- centre la vue sur le bien sélectionné si des coordonnées GPS sont disponibles,
- sinon, se base sur la ville.

L'ensemble renforce l'effet de **cohérence** entre les données backend, la logique métier et l'interface utilisateur.

Analyse de données

Le script analysis.py a pour objectif d'exploiter la base de données ymmo.db afin de :

- obtenir une vision globale du portefeuille,
- analyser les prix par ville,
- identifier les biens les plus grands,
- évaluer le chiffre d'affaires généré par les transactions,
- illustrer la partie "analyse de données" du sujet.

Le script utilise le module sqlite3 pour :

- se connecter à ymmo.db,
- exécuter des requêtes SQL d'agrégation,
- formater les résultats dans la console.

La sortie est purement textuelle (tableaux simplifiés et valeurs clé), ce qui suffit pour une démonstration pédagogique.

Conclusion

Ce projet a permis de concevoir et de développer une solution web complète de gestion immobilière pour Ymmo, intégrant :

- un backend solide,
- un frontend ergonomique,
- et une première couche d'analyse de données.

La partie DEV répond aux objectifs principaux du sujet : modélisation d'un besoin métier réel, mise en place d'une architecture backend orientée services, conception d'une interface web moderne et intégration de traitements de données pour aider à la décision.